

Laboratório de Concorrência · Aula 10

Maria Eduarda Furlan Scomparin · RA 24021296-2 · Turma A

Gerado em 09/10/2026, 20:17:51 · tempo registrado no cronômetro: 0:00

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Sincronização de cooperação é onde uma tarefa precisa esperar a outra ser terminada para ser executada. Já a de competição é onde duas tarefas querem utilizar o mesmo recurso, porém ele não pode ser utilizado ao mesmo tempo.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

2.000.000

1ª execução (n = 1.000.000)

1029321

2ª execução (n = 1.000.000)

1968123

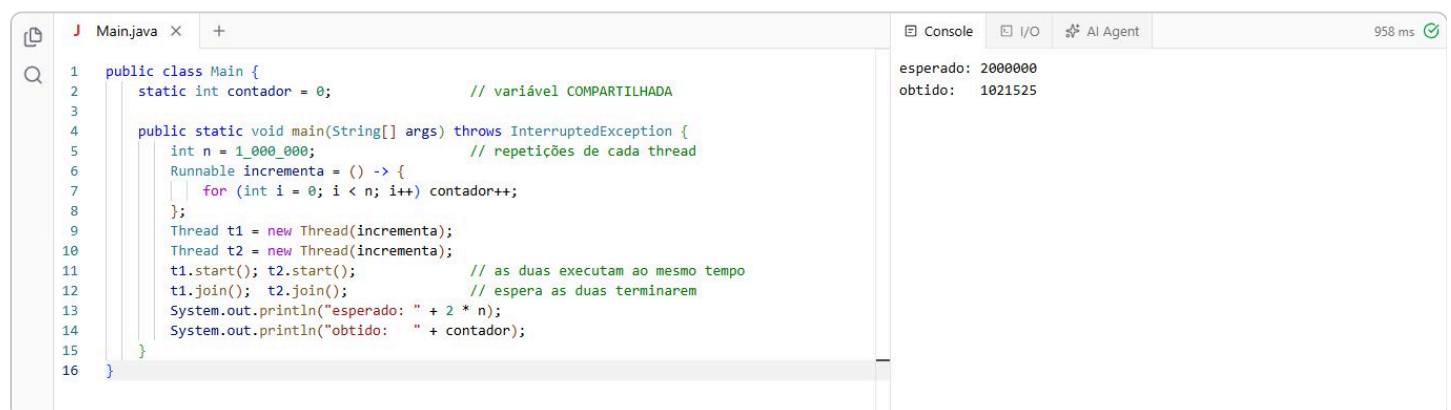
3ª execução (n = 1.000.000)

1021525

Saída do programa

esperado: 2000000

obtido: 1029321



```
1 public class Main {
2     static int contador = 0; // variável COMPARTILHADA
3
4     public static void main(String[] args) throws InterruptedException {
5         int n = 1_000_000; // repetições de cada thread
6         Runnable incrementa = () -> {
7             for (int i = 0; i < n; i++) contador++;
8         };
9         Thread t1 = new Thread(incrementa);
10        Thread t2 = new Thread(incrementa);
11        t1.start(); t2.start(); // as duas executam ao mesmo tempo
12        t1.join(); t2.join(); // espera as duas terminarem
13        System.out.println("esperado: " + 2 * n);
14        System.out.println("obtido: " + contador);
15    }
16 }
```

Console I/O AI Agent 958 ms

esperado: 2000000
obtido: 1021525

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Os valores não bateram com o esperado e não foram iguais nas 3 execuções. Mudaram de uma execução pra outra por conta do contador.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Uma atualização se perde quando duas threads tentam atualizar a mesma variável ao mesmo tempo, sem sincronização. A Thread 2 leu o valor antigo (0) antes de a Thread 1 gravar sua atualização. Por isso, quando a Thread 2 gravou 1, ela sobrescreveu o resultado anterior, sem considerar que a Thread 1 já havia incrementado o contador.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com n = 1000, imagine que duas threads executam 1.000 incrementos cada sobre a mesma variável compartilhada. Se todas as atualizações fossem realizadas corretamente, o resultado esperado seria: 2000. Com n = 1000 o problema pode ficar mais fácil de observar, mas não existe garantia de que o resultado será incorreto em toda execução. A solução é sincronizar a operação completa ou usar uma operação atômica, conforme a necessidade do programa

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

ABABAB → 6

AAABBB → 8

Saída do programa

```
print("Ordens possíveis:", sum(len(v) for v in resultados.values()))
for valor in sorted(resultados):
    ordens = resultados[valor]
    print(f"TOTAL = {valor}: {len(ordens):2d} ordens, por exemplo {ordens[0]}")
```

```
8def executa(ordem):
    # ordem: por exemplo "AABABB"
    134 ms
    AAABBB → 8
    BBBAAB → 7
    ABABAB → 6
    Ordens possíveis: 20
    TOTAL = 4: 9 ordens, por exemplo AABBBB
    TOTAL = 6: 9 ordens, por exemplo AABABB
    TOTAL = 7: 1 ordens, por exemplo BBBAAB
    TOTAL = 8: 1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

Apenas uma ordem dá 8. Há 20 ordens possíveis.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

AABBBB.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acredito que a versão com operação atômica tende a ser a mais rápida e nenhuma incorreta.

Saída do programa

```
1) esperado:          20000000
sem sincronizacao: 1009446 em 2 ms
synchronized:        20000000 em 87 ms
AtomicInteger:       20000000 em 65 ms
```

```
2)esperado:          20000000
sem sincronizacao: 983974 em 2 ms
synchronized:        20000000 em 134 ms
AtomicInteger:       20000000 em 64 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

Nenhuma deu o valor correto, porém a mais próxima foi a 2.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não. Um AtomicInteger em cada saldo não resolve completamente o problema, pois garante a atomicidade de cada operação individual, mas não da transferência inteira. É necessário garantir que o débito de uma conta e o crédito na outra ocorram juntos, usando mecanismos como synchronized, locks ou transações de banco de dados, para evitar inconsistências nos saldos.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não necessariamente. As mensagens "A terminou" e "B terminou" provavelmente não vão aparecer, porque ocorre um deadlock (impasse): a thread A pega o recurso X e espera pelo Y, enquanto a thread B pega o recurso Y e espera pelo X. Como cada uma está esperando um recurso que a outra não libera, nenhuma consegue continuar.

Saídas (antes e depois da correção)

```
1)IMPASSE: 2 threads presas
2) A terminou
   B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

As duas threads ficaram presas por causa de um deadlock (impasse). A thread A tinha o recurso X e esperava pelo Y, enquanto a thread B tinha o recurso Y e esperava pelo X. Como cada uma precisava do recurso que estava com a outra, nenhuma conseguia continuar ou liberar o recurso que já possuía. Assim, as duas ficaram bloqueadas indefinidamente.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Pegar os recursos sempre na mesma ordem elimina o impasse porque impede que uma thread fique esperando por um recurso enquanto outra segura esse recurso e espera por algo que a primeira possui, formando um ciclo de espera. Por exemplo, se todas as threads pegarem primeiro X e depois Y, nenhuma poderá segurar Y enquanto espera por X. Um exemplo real seria um sistema bancário em que duas transferências acessam duas contas ao mesmo tempo: se cada transferência bloquear as contas em uma ordem diferente, pode ocorrer deadlock; se

ambas bloquearem primeiro a conta de menor identificador e depois a de maior identificador, esse ciclo de espera é evitado.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Com 3 trabalhadores: cada um executa 2 tarefas. O tempo esperado é 200.

$2 \times 100 = 200$ ms.

Com 1 trabalhador: ele executa as 6 tarefas em sequência. O tempo esperado é 600.

$6 \times 100 = 600$ ms.

Com 6 trabalhadores: cada um executa uma tarefa ao mesmo tempo. O tempo esperado é 100.

$1 \times 100 = 100$ ms.

Tempo com 1 trabalhador

600ms

Tempo com 3 trabalhadores

200ms

Tempo com 6 trabalhadores

100ms

Saída do programa

```
trabalhador 1 fez a tarefa 2
trabalhador 2 fez a tarefa 3
trabalhador 3 fez a tarefa 1
trabalhador 3 fez a tarefa 6
trabalhador 2 fez a tarefa 5
trabalhador 1 fez a tarefa 4
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador, as 6 tarefas são executadas em sequência, levando aproximadamente 600 ms. Com 3 trabalhadores, cada um executa 2 tarefas, reduzindo o tempo para cerca de 200 ms. Com 6 trabalhadores, cada um pode executar uma tarefa ao mesmo tempo, levando aproximadamente 100 ms. Os tempos reais podem variar um pouco devido ao escalonamento e à sobrecarga do sistema. O programa não precisa de synchronized nem de trava porque as tarefas são distribuídas por um canal (chan), que permite a comunicação segura entre as goroutines, sem que elas precisem compartilhar e modificar diretamente uma variável comum.

Saídas (antes e depois da troca)

```
Antes:
fatal error: all goroutines are asleep - deadlock!
```

```
goroutine 1 [chan send]:
main.main()
    /box/main.go:9 +0x38
exit status 2
```

Depois: 1

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

Por que o canal esta sem espaço e não envia nada.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Acho que não vai compilar.

Saídas (o erro e o resultado com move)

```
1)
  println!("{}", contador);
}
Exit 1
error[E0373]: closure may outlive the current function, but it borrows `contador`, which is
owned by the current function
  → HelloWorld.rs:7:27
   |
7 |     let t = thread::spawn(|| {
   |                               ^^ may outlive borrowed value `contador`
8 |         contador += 1;           // a thread altera a variável de main
   |         ----- `contador` is borrowed here
   |
note: function requires argument type to outlive `static`

2) warning: value captured by `contador` is never read
  → HelloWorld.rs:8:9
   |
8 |         contador += 1;           // a thread altera a variável de main
   |         ^^^^^^^^^^
   |
= help: did you mean to capture by reference instead?
= note: `#[warn(unused_assignments)]` (part of `#[warn(unused)]`) on by default

warning: value assigned to `contador` is never read
  → HelloWorld.rs:8:9
   |
8 |         contador += 1;           // a thread altera a variável de main
   |         ^^^^^^^^^^^^^^^^^^
   |
= help: maybe it is overwritten before being read?

warning: 2 warnings emitted

0
```

Saída do programa

contador = 400000

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Por que a thread copia o numero por conta do move.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

O erro foi descoberto logo no inicio em Java. porém em rust descobriu logo depois do resultado. Assim, fazendo com que java seja melhor e mais rápido.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

B,A,D,C

Saída do programa

```
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A, D, C, B, E, b compilou depois.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer de 10 ms executou bem depois do esperado porque a função ocupado bloqueia a única thread do JavaScript por aproximadamente 300 ms. Enquanto esse cálculo está em execução, o laço de eventos não consegue processar outros callbacks, mesmo que seus timers já tenham vencido.

Em um servidor Node.js, isso é perigoso porque, enquanto um cálculo pesado bloqueia a thread principal, as requisições de outros usuários podem ficar esperando para serem processadas. Isso aumenta a latência e pode causar lentidão para todos os usuários, mesmo que suas requisições sejam simples.

Para evitar isso, cálculos pesados podem ser transferidos para Worker Threads ou processos separados, permitindo que a thread principal continue atendendo o laço de eventos.

Fechamento: Quem impede a corrida?

Java sem sincronização

O compilador

Java com synchronized

O compilador

Go com canais

O programador, com sincronização

Rust com Arc e Mutex

Só existe uma thread

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo.

Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode ocorrer uma condição de corrida: as duas pessoas verificam ao mesmo tempo que há 1 ingresso disponível e ambas conseguem comprá-lo, vendendo dois ingressos para apenas um lugar.

Para evitar isso, é necessário garantir que a verificação da disponibilidade e a reserva do ingresso sejam feitas de forma atômica, por exemplo, usando uma transação no banco de dados com bloqueio adequado ou uma atualização condicional que só permita reservar o ingresso se ainda houver disponibilidade. Assim, apenas uma compra será confirmada e a outra receberá uma mensagem informando que os ingressos acabaram.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

Não sabia que existia tempo de compilação em ms durante a compilação.